KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)
Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A" Grade



Hospital FAQ & Appointment Helper

A Retrieval-Augmented Generation (RAG) Based Hospital Information and Appointment Assistance System

Project ID: PRJ039

Team ID: Team Ignite

Team GitHub Repository: https://github.com/laav-marusu/HospitalFAQ_RAG.git

Live link : https://hospital-react-gamma.vercel.app/

Student Name	Registration Number	Individual GitHub Link
Marusu Lavanya	99230040134	laav-marusu/GEN-AI-Practice
Chintapudi Hasini	99230040095	hasini-chintapudi/GEN-AI-Practice
Anumolu Meghana	99230040136	Anumolu-Meghana/GEN_AI
Chinnabaddila Ujwala	99230040015	ujwalacb3-lab/Gen-Ai-Practice
Maha Sakthi Nivas.K	99230041288	Maha-Sakthi-Nivas/GEN_AI

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of contents:

S.NO	CONTENTS	
1	Introduction	Section 1
2	Problem Statement & Objectives	Section 2
3	System Architecture (RAG Pipeline)	Section 3
4	Implementation Details & Tech Stack	Section 4
5	Results and Testing	Section 5
6	Conclusion & Future Scope	Section 6

1. Introduction

Hospitals provide various healthcare services, including medical consultations, diagnostic facilities, specialist departments, and appointment scheduling. Patients and visitors often need information about hospital departments, doctor availability, consultation fees, outpatient department (OPD) timings, and appointment procedures. Finding this information manually can be time-consuming and inconvenient.

The Hospital FAQ & Appointment Helper is an AI-powered application developed using Retrieval-Augmented Generation (RAG) technology. It helps users obtain hospital-related information through a conversational chatbot. The system retrieves relevant information from a collection of hospital documents and uses a Large Language Model (LLM) to generate clear and understandable answers.

The application uses a vector database to store document embeddings and perform semantic searches. Users can ask questions in natural language instead of searching through multiple documents manually.

In addition to the FAQ chatbot, the application provides an appointment assistance feature that allows users to select hospital departments, choose doctors, select preferred dates and times, and submit appointment requests.

The main purpose of this project is to simplify access to hospital information and improve the convenience of administrative assistance through an AI-based system.

2. Problem Statement & Objectives

Problem Statement

Patients often face difficulties obtaining basic hospital information, such as doctor availability, department details, consultation fees, OPD timings, and appointment procedures. Traditional information systems generally require users to search through multiple documents or contact hospital reception staff to obtain answers.

Searching through lengthy hospital documents manually can consume time and may make it difficult to identify specific information. Basic keyword-based search systems also lack the ability to understand the meaning and context of natural-language questions.

Therefore, an intelligent system is required to provide accurate, context-aware hospital information and simplify the appointment request process.

Objectives

- To develop an AI-powered hospital FAQ assistant using Retrieval-Augmented Generation (RAG).
- To provide answers based on hospital-related documents such as department details, doctor information, consultation fees, and OPD timings.
- To implement document processing and text chunking for efficient information retrieval.
- To generate embeddings using a pre-trained sentence-transformer model.
- To implement FAISS as a vector database for semantic similarity search.
- To integrate a Large Language Model to generate natural-language answers using retrieved information.
- To develop a user-friendly interface using Streamlit.
- To provide an appointment assistance feature for selecting departments, doctors, dates, and preferred time slots.
- To store appointment requests using an SQLite database.
- To reduce incorrect or unsupported answers by restricting responses to the available hospital knowledge base.

3. System Architecture (RAG Pipeline)

The proposed Hospital FAQ & Appointment Helper consists of two major components:

1. Data Ingestion and RAG Processing.
2. Query Processing and Appointment Assistance.

3.1 Data Ingestion Phase

The data ingestion phase prepares hospital documents for semantic search.

The system accepts hospital-related documents in PDF and TXT formats. These documents contain information about hospital facilities, departments, doctors, OPD timings, consultation fees, and appointment guidelines.

The documents are processed through the following steps:

Step 1: Document Loading

Hospital documents are loaded using document loaders such as TextLoader and PyPDFLoader.

Step 2: Text Extraction

The textual information is extracted from the uploaded documents.

Step 3: Text Chunking

The extracted text is divided into smaller sections using the Recursive Character Text Splitter. The chunk size is configured to 500 characters with an overlap of 50 characters to preserve contextual information.

Step 4: Embedding Generation

Each text chunk is converted into a numerical vector using the Sentence Transformers model (all-MiniLM-L6-v2).

Step 5: Vector Database Storage

The generated embeddings and their associated document information are stored in the FAISS vector database for efficient semantic retrieval.

3.2 Query Processing and Generation Phase

When a user submits a question through the hospital chatbot, the following operations take place:

1. The user's question is received through the Streamlit interface.
2. The question is converted into an embedding using the same embedding model.
3. FAISS performs a similarity search to identify the most relevant hospital document chunks.
4. The retrieved information is provided as contextual input to the Large Language Model.
5. The LLM generates a natural-language response based on the retrieved hospital information.
6. The generated answer is displayed to the user along with its supporting document source, where available.

The system is designed to avoid generating unsupported hospital information. If the requested information is unavailable in the knowledge base, the chatbot should inform the user that the information could not be found.

3.3 Appointment Assistance Phase

The appointment assistance component provides a simple interface for submitting hospital appointment requests.

Users can:

- Enter their name and contact number.
- Select a hospital department.
- Choose a doctor from the selected department.
- Select a preferred appointment date.
- Select an available time slot.
- Submit an appointment request.

The appointment details are stored in an SQLite database. The system can display a confirmation message after successfully saving the request.

The appointment component is an administrative feature and does not perform medical diagnosis or treatment recommendations.

4. Implementation Details & Tech Stack

The application uses Python-based technologies and open-source libraries to implement document processing, semantic retrieval, AI-generated responses, and appointment assistance.

Component	Technology Used	Purpose
Programming Language	Python	Core application development
Frontend Framework	Streamlit	Developing the interactive web interface
Orchestration Framework	LangChain	Connecting document loaders, text splitters, and vector stores
Document Processing	PyPDF	Extracting text from PDF documents
Text Processing	Recursive Character Text Splitter	Dividing documents into smaller chunks
Embedding Model	HuggingFace Sentence Transformers (all-MiniLM-L6-v2)	Converting text into numerical vector representations
Vector Database	FAISS	Storing and retrieving document embeddings
Large Language Model	Google Gemini API	Generating natural-language responses using retrieved context
Database	SQLite	Storing appointment information
Development Environment	Visual Studio Code	Writing and executing Python code

4.1 Sample Chunking Configuration

To preserve contextual information during document processing, the Recursive Character Text Splitter is configured with a chunk size of 500 characters and an overlap of 50 characters.

Sample Python configuration:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = text_splitter.split_documents(documents)
```

4.2 Embedding Generation

The Sentence Transformers model converts document chunks into numerical vector representations.

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

4.3 Vector Database Implementation

FAISS is used to create a searchable vector database from the generated document embeddings.

```
from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_documents(
    chunks,
    embeddings
)

vectorstore.save_local("vectorstore")
```

The vector database allows the application to retrieve relevant information based on semantic similarity rather than exact keyword matching.

4.4 Appointment Database

SQLite is used to store appointment request details, including patient name, contact number, department, doctor, preferred date, preferred time, and booking status.

This provides a simple database solution without requiring an external database server.

5. Results and Testing

The Hospital FAQ & Appointment Helper is designed to provide an interactive platform for retrieving hospital information and assisting users with appointment requests.

The system will be evaluated using sample questions based on the hospital knowledge base.

5.1 Sample Testing Questions

Test Case	User Question	Expected Result
1	What departments are available in the hospital?	Display the available hospital departments.
2	Who is the cardiologist?	Retrieve the cardiologist's information.
3	What are the cardiology OPD timings?	Display the relevant OPD schedule.
4	What is the specialist consultation fee?	Retrieve the specialist consultation fee.
5	What documents should I bring for an appointment?	Display the required document information.
6	Is emergency service available?	Retrieve the hospital emergency service information.
7	How can I book an appointment?	Explain the appointment request procedure.
8	What is the hospital address?	Retrieve the hospital location.

5.2 Expected Results

The expected outcomes of the project are:

- The system should successfully load hospital-related PDF and TXT documents.
- The document processing component should extract and split text into meaningful chunks.
- The embedding model should convert document chunks into numerical vectors.
- FAISS should retrieve relevant information for user questions.
- The LLM should generate understandable answers using retrieved document context.
- The application should display relevant hospital information and document sources.
- Users should be able to submit appointment requests through the interface.
- Appointment information should be stored in the SQLite database.

Actual retrieval accuracy, response time, and appointment functionality will be measured after completing and testing the application.

6. Conclusion & Future Scope

Conclusion

The Hospital FAQ & Appointment Helper is an AI-powered application designed to simplify access to hospital-related information using Retrieval-Augmented Generation technology.

By combining document processing, semantic search, vector embeddings, and Large Language Models, the application provides a conversational method for retrieving information from hospital documents.

The integration of FAISS enables efficient semantic retrieval, while the LLM generates natural-language responses based on the retrieved information. The additional appointment assistance feature provides users with a convenient way to submit appointment requests.

Overall, the proposed system aims to reduce the time required to find hospital information and improve the accessibility of administrative services through an intelligent and user-friendly application.

Future Scope

The application can be enhanced with the following features in future versions:

1. **Multilingual Support:** Add support for regional languages such as Tamil, Telugu, and Hindi.
2. **Voice-Based Interaction:** Allow users to ask hospital-related questions using voice input.

3. **Admin Dashboard:** Provide an administrative interface to manage doctors, departments, fees, and appointment requests.
4. **Real-Time Appointment Availability:** Integrate with an authorized hospital scheduling system to display actual available appointment slots.
5. **User Authentication:** Implement secure login and role-based access for administrators and users.
6. **Document Upload:** Allow authorized administrators to upload new hospital documents and update the knowledge base.
7. **Improved Retrieval:** Implement advanced retrieval techniques such as hybrid search and reranking to improve answer relevance.
8. **Appointment Notifications:** Provide appointment confirmation and reminder notifications through authorized communication services.
9. **Source References:** Display the relevant document name and supporting information for chatbot answers.